

IMMO BURUNDI

COMPLETE BACKEND SYSTEM SPECIFICATION

Version 1.0

Project	IMMO BURUNDI
Document	Complete Backend System Specification
Version	1.0
Purpose	Developer implementation document
Platforms	Website + Android + iOS + Admin Dashboard
Languages	French / English / Swahili
Currencies	BIF / USD

Table of Contents

- 1. BACKEND OBJECTIVE 4
- 2. RECOMMENDED ARCHITECTURE 4
- 3. HIGH-LEVEL BACKEND ARCHITECTURE 6
- 4. CORE BACKEND MODULES 6
- 5. DATABASE DESIGN 7
- 6. USERS TABLE 7
- 7. USER ROLES 8
- 8. PERMISSIONS 8
- 9. ROLE PERMISSIONS 9
- 10. USER-SPECIFIC PERMISSIONS 9
- 11. DEPARTMENTS 9
- 12. GEOGRAPHIC STRUCTURE 10
- 13. STAFF PROFILE 10
- 14. PROPERTY TABLE 11
- 15. PROPERTY ID 11
- 16. PROPERTY TYPES 11
- 17. LISTING TYPES 12
- 18. PROPERTY STATUS 12
- 19. PROPERTY VERIFICATION STATUS 12
- 20. PROPERTY DOCUMENTS 12
- 21. FILE STORAGE 13
- 22. PROPERTY PHOTOS 13
- 23. PROPERTY VERIFICATION 13
- 24. VERIFICATION WORKFLOW 14
- 25. VERIFICATION RESULT 14
- 26. VERIFICATION DISCLAIMER 14
- 27. OWNERSHIP AND CONTACT PRIVACY 15
- 28. AGENTS 15
- 29. AGENT PERFORMANCE 16
- 30. PROPERTY VIEWS 16
- 31. VIEW ANALYTICS 16
- 32. FAVORITES 17
- 33. PROPERTY SHARES 17
- 34. ENQUIRIES 17

35. MESSAGING	18
36. WHATSAPP CONTACT	18
37. VISIT BOOKINGS	18
38. VISIT BOOKINGS	18
39. VISIT BOOKING RULES	19
40. RENTAL APPLICATIONS	19
41. RENTAL APPLICATION PRIVACY	19
42. PROMOTION SYSTEM	20
43. PROMOTION ORDERS	20
44. PROMOTION WORKFLOW	20
45. PAYMENT SYSTEM	21
46. PAYMENT REFERENCES	21
47. ACCOUNTING STRUCTURE	21
48. SALES COMMISSIONS	22
49. RENTAL COMMISSIONS	22
50. TRANSACTION RECORD	23
51. REFUNDS	23
52. FINANCIAL ADJUSTMENTS	23
53. MAIN ADMIN FINANCIAL DASHBOARD	23
54. REPORTING SYSTEM	24
55. REPORT EXPORTS	25
56. SEARCH ENGINE	25
57. SEARCH PERFORMANCE	25
58. CURRENCY	26
59. LANGUAGE SYSTEM	26
60. NOTIFICATIONS	27
61. NOTIFICATION PREFERENCES	27
62. ADMIN NOTIFICATIONS	27
63. OFFLINE AGENT MODE	28
64. OFFLINE SYNC	28
65. SYNC CONFLICTS	29
66. IMAGE PROCESSING	29

1. BACKEND OBJECTIVE

The IMMO BURUNDI backend is the central system responsible for:

- User authentication
- User accounts
- Roles and permissions
- Property management
- Property verification
- Agents
- Clients
- Property documents
- Property publication
- Search
- Favorites
- Property views and analytics
- Enquiries
- Visit bookings
- Rental applications
- Promotions
- Payments
- Accounting
- Sales commissions
- Rental commissions
- Notifications
- Messaging
- Reports
- Staff management
- Audit logs
- File/document storage
- Multilingual content
- Currency handling
- Offline agent synchronization
- Security
- Backups
- API access

The backend must be the source of truth for all business-critical information.

2. RECOMMENDED ARCHITECTURE

Recommended stack:

Backend

- Node.js
- TypeScript
- NestJS or another structured TypeScript backend framework
- REST API

- WebSocket support where real-time updates are required

Database

- PostgreSQL

ORM

- Prisma ORM or equivalent

Cache

- Redis

File storage

Use object storage such as:

- AWS S3
- Cloudflare R2
- MinIO
- equivalent secure object storage

Authentication

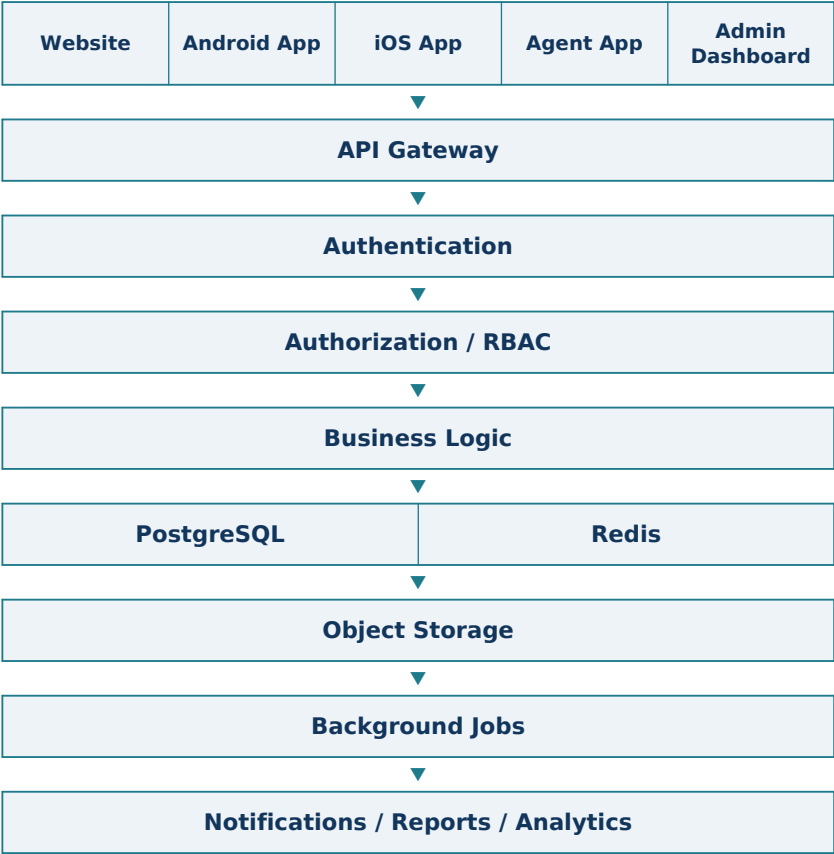
- JWT access tokens
- Refresh tokens
- Secure password hashing
- Optional OTP authentication

Background jobs

Use Redis/BullMQ or equivalent for:

- Emails
- SMS
- WhatsApp notifications
- Payment verification
- Reports
- Image processing
- Analytics aggregation
- Expired bookings
- Expired promotions
- Scheduled notifications

3. HIGH-LEVEL BACKEND ARCHITECTURE



4. CORE BACKEND MODULES

Create separate backend modules/services for:

1. Authentication
2. Users
3. Roles
4. Permissions
5. Staff
6. Departments
7. Geographic areas
8. Properties
9. Property media
10. Property documents
11. Verification
12. Agents
13. Clients
14. Favorites
15. Enquiries
16. Visits
17. Rental applications
18. Promotions

- 19. Payments
- 20. Accounting
- 21. Sales commissions
- 22. Rental commissions
- 23. Notifications
- 24. Messaging
- 25. Analytics
- 26. Reports
- 27. Audit logs
- 28. Search
- 29. File storage
- 30. Settings
- 31. Localization
- 32. Currency
- 33. Offline synchronization
- 34. Security
- 35. System administration

5. DATABASE DESIGN

PostgreSQL should be the primary relational database.

6. USERS TABLE

users

Suggested fields:

id	phone	phone_verified_at
uuid	password_hash	last_login_at
first_name	preferred_language	created_at
last_name	status	updated_at
email	email_verified_at	deleted_at

Status:

PENDING	DISABLED
ACTIVE	LOCKED
SUSPENDED	DELETED

Never store plain-text passwords.

7. USER ROLES

roles

id	is_system_role
name	created_at
description	updated_at

Examples:

MAIN_ADMIN	ACCOUNTANT	BUYER
CHIEF	MODERATOR	TENANT
DEPARTMENT_HEAD	SUPPORT_OFFICER	INVESTOR
PROVINCIAL_ADMIN	MARKETING_OFFICER	CLIENT
COMMUNE_ADMIN	FIELD_AGENT	GUEST
REGIONAL_SUPERVISOR	PROPERTY_OWNER	
VERIFICATION_OFFICER	LANDLORD	

8. PERMISSIONS

permissions

id	resource
code	action
name	created_at
description	

Examples:

properties.view	properties.approve	properties.archive
properties.create	properties.reject	properties.delete
properties.update	properties.publish	
properties.submit	properties.unpublish	

verification.view	verification.approve
verification.create	verification.reject
verification.review	verification.generate_code

accounting.view	accounting.reconcile
accounting.create	accounting.refund
accounting.update	accounting.export

reports.view
reports.export
reports.financial
reports.global

audit.view
audit.export

9. ROLE PERMISSIONS

role_permissions
role_id
permission_id
scope
created_at

Scopes:

GLOBAL	TEAM
DEPARTMENT	OWN
PROVINCE	ASSIGNED
COMMUNE	PUBLIC

10. USER-SPECIFIC PERMISSIONS

user_permissions

Allows Main Admin to grant a specific permission without changing the person's primary role.

Fields:

id	commune_id	reason
user_id	department_id	created_at
permission_id	starts_at	revoked_at
scope	expires_at	
province_id	granted_by	

Expired permissions must automatically stop working.

11. DEPARTMENTS

departments

Examples:

Management	Sales	Operations
Verification	Rentals	Customer Support
Accounting	Marketing	Technology

Fields:

id	status
name	created_at
description	updated_at
head_user_id	

12. GEOGRAPHIC STRUCTURE

Create:

provinces
communes
zones

Relationships:

Province
+-- Commune
+-- Zone

This is important for geographic permissions.

13. STAFF PROFILE

staff_profiles		
id	address	start_date
user_id	department_id	internal_notes
staff_code	province_id	created_at
photo_url	commune_id	updated_at
phone	job_title	
email	employment_status	

Sensitive fields must have restricted authorization.

14. PROPERTY TABLE

properties

Core fields:

id	surface_area	owner_user_id
uuid	bedrooms	landlord_user_id
property_code	bathrooms	agent_id
title	rooms	created_by
description	floors	approved_by
property_type	parking_spaces	published_by
listing_type	year_built	published_at
status	province_id	sold_at
verification_status	commune_id	rented_at
verification_level	zone_id	archived_at
price	latitude	created_at
currency	longitude	updated_at
is_negotiable	location_precision	deleted_at

15. PROPERTY ID

Every property receives an immutable unique ID.

Example:

BDI-BJM-KAM-2026-0001

Format:

BDI - [PROVINCE] - [COMMUNE] - [YEAR] - [SEQUENCE]

The ID:

- Is generated by backend
- Cannot be changed by users
- Is globally unique
- Appears on property pages
- Appears on documents
- Appears in enquiries
- Appears in bookings
- Appears in payment references where applicable

16. PROPERTY TYPES

Support:

HOUSE	OFFICE	FARM
APARTMENT	SHOP	INDUSTRIAL
VILLA	WAREHOUSE	OTHER
LAND	HOTEL	
COMMERCIAL	GUEST_HOUSE	

System should allow Main Admin to add new types later.

17. LISTING TYPES

SALE	AUCTION
RENT	INVESTMENT
LEASE	

18. PROPERTY STATUS

DRAFT	APPROVED	RENTED
SUBMITTED	PUBLISHED	ARCHIVED
UNDER_REVIEW	REJECTED	SUSPENDED
NEEDS_CORRECTION	SOLD	

19. PROPERTY VERIFICATION STATUS

NOT_VERIFIED	FULLY_VERIFIED
PARTIAL	EXPIRED
VERIFIED	SUSPENDED

Public visual system:

- Red = Not Verified
- Yellow = Partial
- Green = Verified
- Special badge = Fully Verified

20. PROPERTY DOCUMENTS

property_documents

id	issued_by	verified_at
property_id	issue_date	verification_notes
document_type	expiry_date	created_at
file_id	verification_status	updated_at
document_number	verified_by	

Document types:

LAND_TITLE	SALE_AGREEMENT
TOP	PROPERTY_TAX
OWNER_ID	OTHER

Private documents must not be publicly accessible.

21. FILE STORAGE

files

id	mime_type	visibility
uuid	size	created_at
original_name	checksum	deleted_at
storage_key	uploaded_by	

Visibility:

PRIVATE
INTERNAL
RESTRICTED
PUBLIC

Never expose raw storage URLs for private documents.

Use temporary signed URLs.

22. PROPERTY PHOTOS

property_media

id	sort_order
property_id	caption
file_id	is_primary
media_type	created_at

Media types:

IMAGE
VIDEO
FLOOR_PLAN
DOCUMENT_PREVIEW

23. PROPERTY VERIFICATION

verification_requests

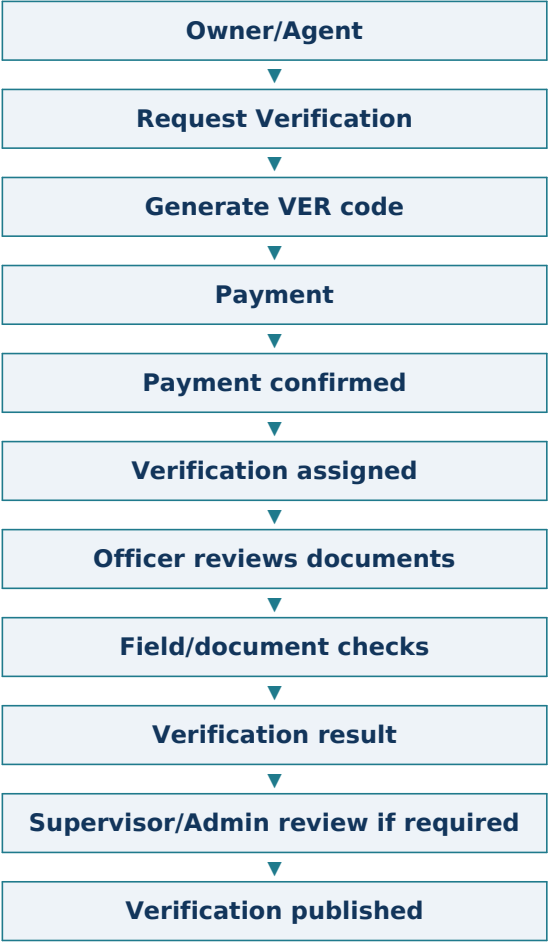
id	status	completed_by
verification_code	priority	result
property_id	requested_at	notes
requested_by	started_at	created_at
assigned_officer	completed_at	updated_at

Verification code example:

VER-BDI-2026-000001

The code must be unique.

24. VERIFICATION WORKFLOW



25. VERIFICATION RESULT

Store individual checks.

`verification_checks`

<code>id</code>	<code>status</code>
<code>verification_request_id</code>	<code>notes</code>
<code>document_id</code>	<code>checked_by</code>
<code>check_type</code>	<code>checked_at</code>

Status:

PASSED
FAILED
PARTIAL
NOT_APPLICABLE

26. VERIFICATION DISCLAIMER

The backend must store the version of the disclaimer displayed at verification time.

This protects historical records if the website disclaimer is later changed.

Every verification record should contain:

```
disclaimer_version
disclaimer_accepted_at
```

The public property page must display the applicable disclaimer.

Core meaning:

Verification is based on documents and information presented at the time of review. It is not an absolute guarantee against future disputes, hidden claims, fraud, registry errors, or changes occurring after verification.

27. OWNERSHIP AND CONTACT PRIVACY

Store separately:

```
owner_user_id
owner_phone_private
owner_email_private
owner_address_private
```

These fields are never automatically exposed publicly.

The public page should normally use:

Contact Agent	Send Message
Call	Request Visit
WhatsApp	

rather than exposing private owner information.

28. AGENTS

```
agents
```

id	license_number	total_rentals
user_id	photo	status
agent_code	bio	top_agent
province_id	rating	created_at
commune_id	total_deals	updated_at
agency_name	total_sales	

Example:

AG-BJM-0001

29. AGENT PERFORMANCE

agent_performance

Track:

properties_submitted	properties_rented	views_generated
properties_approved	verification_requests	enquiries_generated
properties_published	successful_deals	visit_bookings
properties_sold	commission_total	conversion_rate

Agents should only see their own detailed performance unless authorized.

30. PROPERTY VIEWS

Create:

property_views

Fields:

id	anonymous_id	city_approximate
property_id	device_type	referrer
user_id nullable	browser	ip_hash
session_id	country	viewed_at

Do not expose raw IP addresses unnecessarily.

31. VIEW ANALYTICS

Backend should calculate:

- Total views
- Unique visitors
- Views today
- Views this week
- Views this month
- Views over 90 days
- Logged-in viewers
- Anonymous viewers
- Traffic sources
- Device types
- Approximate geographic distribution

Owner/agent analytics must not reveal viewer identity merely because they viewed the property.

32. FAVORITES

favorites

id
user_id
property_id
created_at

Unique constraint:

user_id + property_id

33. PROPERTY SHARES

property_shares

id	platform
property_id	created_at
user_id nullable	

Platforms:

WHATSAPP	COPY_LINK
FACEBOOK	OTHER
INSTAGRAM	

34. ENQUIRIES

enquiries

id	agent_id	updated_at
property_id	message	closed_at
sender_user_id	status	
recipient_user_id	created_at	

Status:

NEW	RESPONDED
OPEN	CLOSED
IN_PROGRESS	

35. MESSAGING

conversations

```
id
property_id
created_at
updated_at
```

conversation_participants

```
conversation_id
user_id
joined_at
```

messages

```
id                                attachment_id
conversation_id                  read_at
sender_id                       created_at
message                         deleted_at
```

Implement abuse/report functionality.

36. WHATSAPP CONTACT

Generate a controlled WhatsApp link.

Message example:

Hello, I am interested in property BDI-BJM-KAM-2026-0001.

Do not expose private owner information through the frontend unnecessarily.

37. VISIT BOOKINGS

visit_sessions

```
id                end_time                booking_deadline
property_id       capacity
date             booked_count
start_time        status
created_by
created_at
updated_at
```

38. VISIT BOOKINGS

visit_bookings

```
id                number_of_people    confirmed_at
visit_session_id booking_reference    cancelled_at
user_id          notes
status          created_at
```

Statuses:

PENDING	NO_SHOW
CONFIRMED	COMPLETED
CANCELLED	

39. VISIT BOOKING RULES

Backend must enforce:

- User must be logged in
- Session must be active
- Booking deadline must not have passed
- Capacity must not be exceeded
- Duplicate booking must be prevented
- Cancelled booking frees capacity
- Countdown is calculated from server time
- Time zone is explicitly stored
- Booking reference is unique

Example:**VIS-BDI-2026-000001**

40. RENTAL APPLICATIONS

rental_applications

id	address	move_in_date
application_code	email	status
property_id	total_occupants	landlord_notes
applicant_id	number_of_children	created_at
full_name	occupation	updated_at
phone	advance_available	

Status:

SUBMITTED	ACCEPTED
UNDER_REVIEW	REJECTED
SHORTLISTED	WITHDRAWN

41. RENTAL APPLICATION PRIVACY

Applicants must never see:

- Other applicants
- Other applicants' documents
- Other applicants' phone numbers
- Internal landlord notes

Landlords/authorized staff only see information necessary for application review.

42. PROMOTION SYSTEM

promotion_packages

id	price	created_at
name	currency	updated_at
description	features	
duration_days	status	

Examples:

Featured 7 Days

Homepage Featured

Featured 14 Days

Premium Placement

Featured 30 Days

43. PROMOTION ORDERS

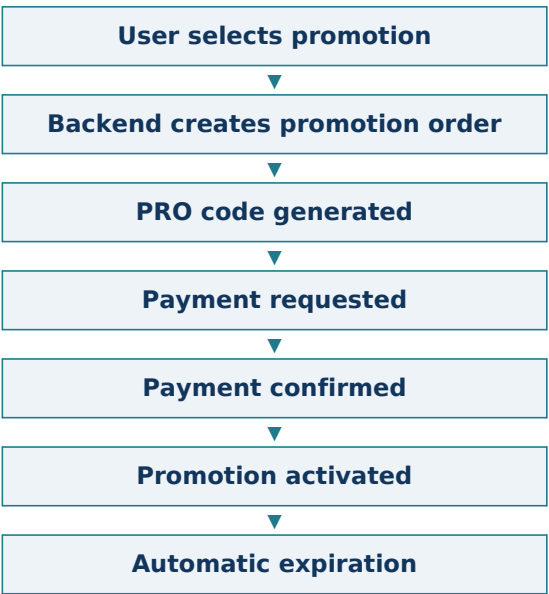
promotion_orders

id	requested_by	starts_at
promotion_code	amount	ends_at
property_id	currency	created_at
package_id	status	updated_at

Example:

PRO-BDI-2026-000001

44. PROMOTION WORKFLOW



No frontend manipulation should be able to activate a paid promotion without backend confirmation.

45. PAYMENT SYSTEM

payments

id	amount	paid_at
payment_reference	currency	confirmed_by
payer_id	payment_method	created_at
category	status	updated_at
related_entity_type	provider	
related_entity_id	provider_transaction_id	

Categories:

VERIFICATION	REFUND
PROMOTION	ADJUSTMENT
SALE_COMMISSION	OTHER
RENTAL_COMMISSION	

46. PAYMENT REFERENCES

Every payment-related transaction must have a unique reference.

Examples:

VER-BDI-2026-000001
PRO-BDI-2026-000001
COMS-BDI-2026-000001
COMR-BDI-2026-000001

Payment references must never be reused.

47. ACCOUNTING STRUCTURE

Do NOT combine everything into one generic revenue number.

Separate ledgers:**Verification Revenue**`verification_payments`**Promotion Revenue**`promotion_payments`**Sales Commissions**`sales_commissions`**Rental Commissions**`rental_commissions`**Refunds**`refunds`**Adjustments**`financial_adjustments`

48. SALES COMMISSIONS

`sales_commissions`

<code>id</code>	<code>buyer_id</code>	<code>due_date</code>
<code>commission_code</code>	<code>sale_amount</code>	<code>paid_at</code>
<code>property_id</code>	<code>commission_rate</code>	<code>created_at</code>
<code>transaction_id</code>	<code>commission_amount</code>	<code>updated_at</code>
<code>agent_id</code>	<code>currency</code>	
<code>seller_id</code>	<code>status</code>	

Example:`COMS-BDI-2026-000001`

49. RENTAL COMMISSIONS

`rental_commissions`

<code>id</code>	<code>agent_id</code>	<code>due_date</code>
<code>commission_code</code>	<code>rent_amount</code>	<code>paid_at</code>
<code>property_id</code>	<code>commission_rate</code>	<code>created_at</code>
<code>rental_application_id</code>	<code>commission_amount</code>	<code>updated_at</code>
<code>landlord_id</code>	<code>currency</code>	
<code>tenant_id</code>	<code>status</code>	

Example:`COMR-BDI-2026-000001`

50. TRANSACTION RECORD

Create a general:

transactions

id	seller_id	currency
transaction_code	tenant_id	status
property_id	landlord_id	transaction_date
transaction_type	agent_id	created_at
buyer_id	amount	updated_at

Types:

SALE
RENTAL
LEASE

51. REFUNDS

refunds

id	amount	approved_by
refund_reference	currency	processed_by
payment_id	reason	processed_at
requested_by	status	created_at

Statuses:

REQUESTED	REJECTED
UNDER_REVIEW	PROCESSED
APPROVED	

Refund history must never be deleted.

52. FINANCIAL ADJUSTMENTS

financial_adjustments

id	amount	approved_by
adjustment_reference	currency	created_at
category	reason	
related_record	created_by	

Large or sensitive adjustments should support dual approval.

53. MAIN ADMIN FINANCIAL DASHBOARD

Only Main Admin sees the complete global financial dashboard by default.

Display:

Total Revenue	Rental Commission Revenue	Pending Payments
Verification Revenue	Refunds	Failed Payments
Promotion Revenue	Adjustments	Outstanding Commissions
Sales Commission Revenue	Net Revenue	

Filters:

- Today
- This week
- This month
- This year
- Custom dates
- Province
- Commune
- Agent
- Payment category
- Currency

54. REPORTING SYSTEM

reports

Generate:**Property reports**

- New listings
- Published
- Sold
- Rented
- Archived
- Verification status

Agent reports

- Activity
- Deals
- Sales
- Rentals
- Performance

Financial reports

- Revenue
- Payments
- Refunds
- Commissions

User reports

- Registrations

- Active users
- Suspended users
- Clients

Analytics reports

- Views
- Unique visitors
- Enquiries
- Favorites
- Shares
- Visits

55. REPORT EXPORTS

Allow authorized users to export:

- CSV
- Excel
- PDF

Exports must be permission-controlled.
Every sensitive export should create an audit event.

56. SEARCH ENGINE

Backend search must support:

property ID	commune	bathrooms
verification code	zone	agent
promotion code	property type	verification status
commission code	listing type	featured
transaction code	price	date
location	surface	
province	bedrooms	

57. SEARCH PERFORMANCE

For PostgreSQL:

- Add indexes to frequently searched columns
- Use full-text search where appropriate
- Consider PostgreSQL "tsvector"
- Use trigram search for names/titles
- Use pagination
- Never return thousands of records at once

For very large scale, introduce Elasticsearch/OpenSearch later.

58. CURRENCY

Supported currencies:

BIF
USD

Every financial record must store its original currency.

Never overwrite the original transaction currency because an exchange rate changed.

Store:

amount	converted_currency
currency	rate_source
exchange_rate	rate_date
converted_amount	

Converted amounts should be treated as estimates unless an official accounting rate is being used.

59. LANGUAGE SYSTEM

Supported:

FR
EN
SW

Backend should return translation keys or localized content.

Example:

property.status.published

Frontend translates it according to language.

For editable content, support:

title_fr
title_en
title_sw

description_fr
description_en
description_sw

60. NOTIFICATIONS

notifications

id	message	read_at
user_id	data	created_at
type	channel	
title	status	

Channels:

IN_APP	WHATSAPP
EMAIL	PUSH
SMS	

Examples:

- Property approved
- Verification completed
- Payment received
- Visit confirmed
- Visit cancelled
- Rental application updated
- Promotion activated
- New enquiry
- New message

61. NOTIFICATION PREFERENCES

notification_preferences

Users can control available notification channels.

Examples:

new_message	visit
property_status	promotion
payment	marketing

Critical security notifications should not be disabled.

62. ADMIN NOTIFICATIONS

Admin notifications include:

- New property submitted
- Verification requested
- Suspicious property
- Payment received
- Failed payment
- Refund requested

- New rental application
- New visit booking
- Agent issue
- System error

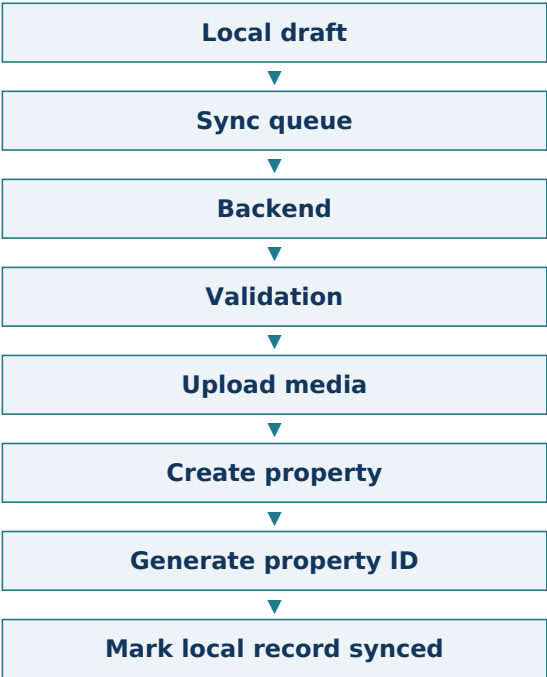
63. OFFLINE AGENT MODE

Agent mobile application must support offline property creation.

Local device stores:

draft property	GPS
photos	owner information
documents	timestamps

When internet returns:



64. OFFLINE SYNC

Every offline record requires:

local_id	sync_attempts
device_id	last_sync_error
created_offline_at	server_id
sync_status	

Statuses:

PENDING	FAILED
SYNCING	CONFLICT
SYNCED	

65. SYNC CONFLICTS

If two versions conflict:

CONFLICT

Do not silently overwrite important information.

Backend should return the conflict to the app for resolution.

66. IMAGE PROCESSING

When an image is uploaded:

1. Validate fi